CSC 3204: AUTOMATA THEORY

Lecture VII: Transition Graphs

Refresher

Considering the alphabet $\sum = \{a,b\}$:

- Build a FA that accepts only the word Λ.
- Build a FA that accepts only the word baa.
- Build a FA that accepts the language described by the following regex:

  $(a + b)^*a$

- Build a FA that accepts the language described by the following regex:

  $(a + ba^*ba^*b)^+$

Transition Graph for "baa"

- The FA accepting "baa" required 5 states since a FA can read only one letter from the input string at a time.
- A Transition Graph (TG) removes this restriction and can read either one or two or more letters of the input string at a time.
- Since a TG can consume many letters at a time, how can we build a TG with even fewer states that accepts "baa"?

How a TG works

- Presented with an input string at the start state
- Reads input string *substring by substring* beginning with the leftmost substring
- If there is a substring that matches at least one transition the input string will transit to a new state (where the next substring will be read)
- The sequence ends when the last input substring has been read
- If the end occurs at a final state then the input string has been accepted by the TG, else the string is rejected.

How many letters to read at a time?

- If we assume that a TG can read one or two letters at a time, then a TG with two states can be built to recognize all words that contain a double letter.
- Build this TG.
- This TG requires us to exercise some choice in its running – we must decide how many letters to read from the input string each time we go back for more. This is a significant decision!
- e.g. The input string "baa" has two paths through the TG, one resulting in acceptance and the other in failure, depending on how we choose to consume the input.

How many letters to read at a time?

- For the input string "baa", if we chose to read the first two letters i.e. ba at the start of string traversal, we could not get to a final state. The processing of this string would break down at this point.
- When there is no edge leaving a state corresponding to the group of input letters that have been read while at that state, we say that the input crashes.
- Crashing signifies that the input is not accepted, but for a different reason rather than ending in a non-final state.

Transition path

- A successful path through a transition graph is a series of edges forming a path beginning at some start state (there may be several) and ending at a final state.
- If we concatenate in order the strings of letters that label each edge in the path, we produce a word that is accepted by the machine.
- If presented with a particular string of *a*'s and *b*'s to run on a given TG, we must decide how to break (factor) the word into substrings that might correspond to the labels of edges in a path.

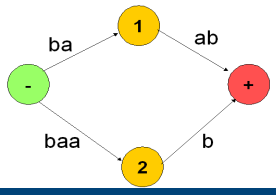
Transition Graphs

- Like regular expressions and finite automata, transition graphs (TGs) may be used to define languages
- TGs are non-deterministic!
- TGs relax some of the rules used to define finite automata

Recap: Transition Graphs

- Like regular expressions and finite automata, transition graphs (TGs) may be used to define languages
- TGs are non-deterministic!
- TGs relax some of the rules used to define deterministic finite automata
- Input differs: can accept substrings as input
- Definition of acceptance differs: A string is accepted by a TG if there is *some* way it could be processed so as to arrive at a final state. Note: there may be ways in which this string does not get to a final state, but such failures are ignored.

An example transition graph



- Is the word baab accepted by this machine? How?
- What of aaa?
- Unlike DFAs where there exists a unique path through the machine for every input string, a TG allows some strings with no paths at all, while some strings have several possible paths.

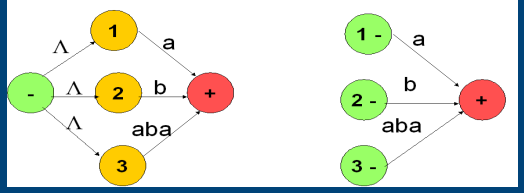
Definition

- A transition graph consists of:
  - A finite set of states at least one of which is designated as the start state (-) and some (maybe none) are designated as final states (+)
  - An alphabet Σ of possible input letters from which input strings are formed.
  - A finite set of transitions that show how to go from one state to another based on reading specified substrings of input letters (including Λ)
- TGs were invented by John Myhill in 1957 to simplify the proof of the following theorem:
  Any language that can be defined by a Regular Expression or a Finite Automaton or a Transition Graph can be defined by all three methods.

Implication of allowing Λ transitions

- If a transition can be labelled with Λ then we have to relax the condition that there can only be one start state!
- The following TGs are equivalent as language acceptors!

Illustration



Points to Note

- Start and final state
- Labelling of transitions with substrings
- Non-determinism
- Ease of development is a trade-off with difficulties in execution
- Transitions can be labelled with Λ
- Every DFA is also a TG. However, not every TG satisfies the definition of a DFA!

Examples of TGs

- Considering the alphabet ∑ = {a,b}, build a TG that:
  1) Accepts nothing, not even the null string Λ.
  2) Accepts only the string Λ.
  3) Accepts only the words Λ, baa and *abba*.
  4) Accepts the language defined by (a + b)*b. What is the equivalent FA that accepts this language?
  5) Accepts the language EVEN-EVEN over the alphabet {a,b}
  6) Accepts the language of all words that begin and end with different letters over {a,b}